



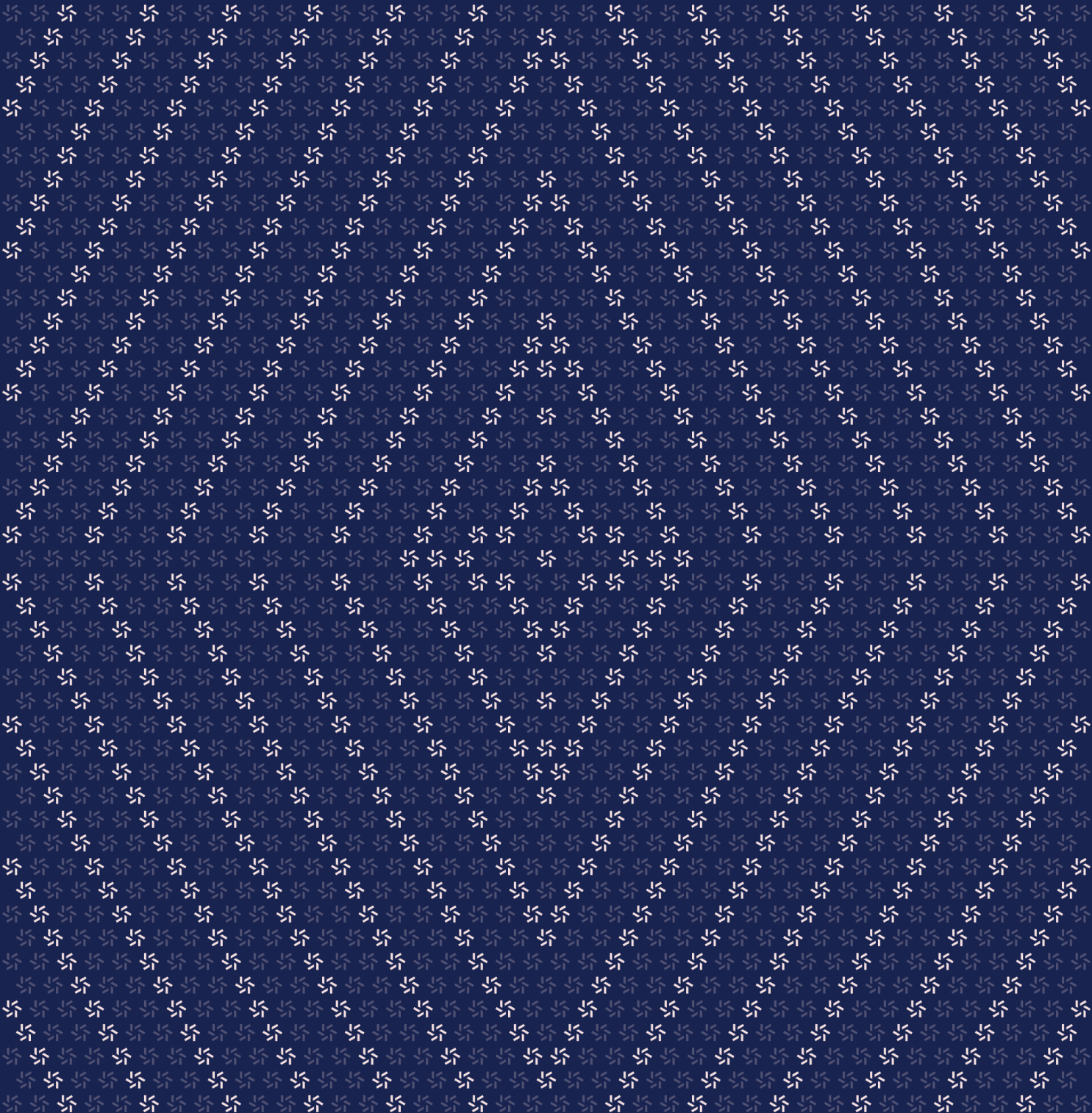
May 5, 2026

Prepared for
AFX

Prepared by
Syed Faraz Abrar
Filipe Alves
Katerina Belotskaia
Nipun Gupta
Zellic

AFX Bridge

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About AFX Bridge	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. Attacker can drain a victim's preapproved USDC via dust deposits	11
3.2. Lack of test coverage	14
3.3. Replayable changeValidatorSet signature can defer rotation finalization indefinitely	18
3.4. Orchestrator is not aware of pausable functionality on Arbitrum	21
3.5. Partially voted withdrawals can stay stuck in the pending state indefinitely	23
3.6. Bridge-address updates can strand pending transfers	25
3.7. Attacker can deposit a victim's preapproved USDC to the Cosmos chain without consent	29

3.8.	Validator rotation can double-count bridge votes	32
3.9.	Relayer vote caches grow without bound	35
3.10.	Multiple issues related to the Bridge contract address update functionality	38
3.11.	Nil validator in bridge vote handling can panic	39
4.	Discussion	41
4.1.	Missing access to the full codebase	42
4.2.	Signature checks missing from two bridge vote-related messages	42
4.3.	Possible quorum-check mismatch between the bridge module and Bridge contract	43
4.4.	Bridge contract hashing can panic in the EndBlocker	43
5.	System Design	44
5.1.	Component: Orchestrator	45
5.2.	Component: Deposits	47
5.3.	Component: Withdrawals	49
5.4.	Component: Bridge contract address change	51
6.	Assessment Results	52
6.1.	Disclaimer	54

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for AFX from April 29th to May 4th, 2026. During this engagement, Zellic reviewed AFX Bridge's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any potential signature replays leading to loss of funds from the bridge?
 - Could an attacker withdraw funds from the bridge without burning tokens on the other chain?
 - Could an attacker steal funds from other users' wallets due to infinite approvals?
 - Are there any-chain halting concerns on the Cosmos side?
 - Are there any issues leading to tokens being stuck on the bridge?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

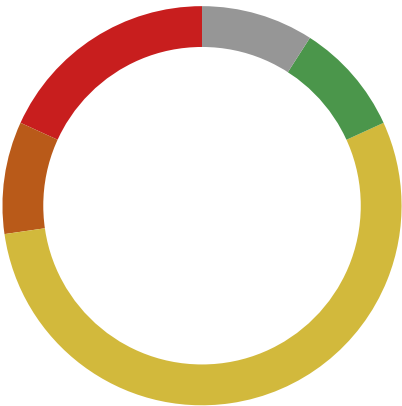
During our assessment on the scoped AFX Bridge contracts, we discovered 11 findings. Two critical issues were found. One was of high impact, six were of medium impact, one was of low impact, and the remaining finding was informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of AFX in the Discussion section ([4. ↗](#)).

Based on the number and severity of findings identified during this engagement, we recommend a comprehensive reassessment of the codebase following remediation efforts; see section [6](#).

Breakdown of Finding Impacts

Impact Level	Count
Critical	2
High	1
Medium	6
Low	1
Informational	1



2. Introduction

2.1. About AFX Bridge

AFX contributed the following description of AFX Bridge:

AFX is a high-performance decentralized perpetual contract exchange built on our own Layer 1 blockchain. It combines on-chain verifiable settlement with a CEX-grade trading experience: sub-second execution, deep liquidity, and transparent risk management.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. 7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

AFX Bridge Contracts

Types	Solidity, Go
Platforms	EVM-compatible, Cosmos
Target	afx-bridge
Repository	https://github.com/afx-dex/afx-bridge
Version	a5537a8e85793d2e6788401dd8c406152b5a4e7b
Programs	contracts/bridge/src/bridge.sol contracts/bridge/src/signature.sol dex-node/x/bridge/**

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.7 person-weeks. The assessment was conducted by four consultants over the course of three calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
↗ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Syed Faraz Abrar
↗ Security Engineer
faith@zellic.io ↗

Filipe Alves
↗ Security Engineer
filipe@zellic.io ↗

Katerina Belotskaia
↗ Security Engineer
kate@zellic.io ↗

Nipun Gupta
↗ Security Engineer
nipun@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

April 29, 2026 Start of primary review period

May 4, 2026 End of primary review period

3. Detailed Findings

3.1. Attacker can drain a victim's preapproved USDC via dust deposits

Target	Bridge		
Category	Business Logic	Severity	Critical
Likelihood	Medium	Impact	Critical

Description

The orchestrator silently drops deposits below 5 USDC, but the on-chain Bridge contract enforces no minimum. Because `batchedDepositWithApproval` is permissionlessly callable by any address on behalf of any approver, an attacker can drain a victim's preapproved allowance into the bridge in many subminimum deposits that are each accepted on chain but ignored by the orchestrator.

The 5 USDC minimum is a constant in the orchestrator:

```
const (
    minDepositAmount = 5_000_000
    // [...]
)
```

It is applied in two places. First, `processArbitrumDeposit` silently logs and discards any deposit below the threshold without storing it:

```
if deposit.Value.Int64() < minDepositAmount {
    o.log.Info("processArbitrumDeposit", "deposit below min amount, ignored",
        deposit.Value.Int64(), "min", minDepositAmount)
    return
}
```

Second, the `/unfinished_deposits` HTTP endpoint polled by the Cosmos-side relayer also excludes subminimum deposits:

```
if item.Raw.Value.Int64() < minDepositAmount {
    continue
}
```

As a result, the Cosmos bridge module's relayer (`relayer.go:141`, which fetches from `/unfinished_deposits`) never sees these deposits and never credits the depositor on chain.

The on-chain Bridge contract, however, imposes no such restriction —

batchedDepositWithApproval accepts any amount and any caller:

```
function batchedDepositWithApproval(
    DepositWithApproval[] memory deposits
) external nonReentrant whenNotPaused {
    uint64 end = uint64(deposits.length);
    for (uint64 idx; idx < end; idx++) {
        depositWithApproval(deposits[idx].user, deposits[idx].usd);
    }
}
```

An attacker exploits the mismatch as follows:

1. A victim approves the Bridge contract for any amount of USDC as part of the normal deposit flow.
2. The attacker calls batchedDepositWithApproval with many entries of {user: victim, usd: 4_999_999} (just under the 5 USDC minimum), repeating until the victim's allowance is exhausted.
3. Each call executes transferFrom(victim, bridge, 4_999_999) successfully and emits a deposit event on chain.
4. Because every deposit is below minDepositAmount, the orchestrator drops them in processArbitrumDeposit and excludes them from /unfinished_deposits. The Cosmos relayer never processes them.
5. The victim's USDC is locked in the Bridge contract with no corresponding credit on the Cosmos chain.

The attack requires no privileged access and is inexpensive to execute on Arbitrum, where gas costs are low.

This finding chains with the High-severity finding for the permissionless batchedDepositWithApproval (see Finding 3.7.7). When deposits are at or above the 5 USDC minimum, the victim at least receives a Cosmos credit and can recover via the withdrawal flow. The Critical severity here arises specifically from subminimum amounts, which leave the victim with no credit on either chain.

Impact

The victim permanently loses the full preapproved USDC allowance. The funds are held by the Bridge contract but are never credited on the Cosmos side. Recovery requires bridge operators to manually construct and validator-sign a withdrawal on the victim's behalf.

Recommendations

Enforce the minimum deposit at the contract level. Add a check in `depositWithApproval` (and symmetrically in `depositWithPermit`) so that the transaction reverts for subminimum amounts:

```
uint64 public constant MIN_DEPOSIT_AMOUNT = 5_000_000;

function depositWithApproval(address user, uint64 usd) private {
    require(usd >= MIN_DEPOSIT_AMOUNT, "Deposit below minimum");
    // ...
}
```

This ensures consistency between on-chain acceptance and off-chain processing and closes the attack surface entirely.

Additionally, consider restricting `batchedDepositWithApproval` to a permissioned caller (e.g., an orchestrator address). Allowing arbitrary callers to initiate transfers on behalf of any approver is an unnecessary permission, and restricting it provides defense in depth.

Remediation

This issue has been acknowledged by AFX, and a fix was implemented in commit [3466c704](#).

3.2. Lack of test coverage

Target	Project-wide		
Category	Protocol Risks	Severity	Critical
Likelihood	N/A	Impact	Critical

Description

No security-critical component of the system has any automated tests.

Zero tests in the Bridge Solidity contract

The file `contracts/bridge/test/bridge.t.sol` contains only an empty `setUp()` with no test functions. None of the contract's state-mutating logic is exercised, including the following:

- Deposit paths — `batchedDepositWithApproval` and `batchedDepositWithPermit`, including failure modes (`FailedApprovalDeposit`, `FailedPermitDeposit`) and the missing minimum-amount enforcement (see related Critical finding, section [3.1.7](#))
- Withdrawal life cycle — `batchedCreateWithdrawals`, `batchedFinalizeWithdrawals`, and `rejectWithdrawals`, including dispute-period enforcement, double-finalization prevention, and cold-validator veto
- Validator set governance — `changeValidatorSet` and `finalizeValidatorSetChange`, including epoch ordering and signature quorum edge cases
- Emergency pause system — `voteEmergencyLock`, `unvoteEmergencyLock`, `emergencyUnlock`, and `onlyUnlock`, including the locker-threshold autopause trigger
- Access control — `changeLocker` and `changeFinalizer`, including the hot-/cold-validator distinction for grant versus revoke
- Signature verification — the `checkValidatorSignatures` ordering invariant (out-of-order signatures must fail to reach quorum even when individually valid)

Zero tests in the Cosmos bridge module (`dex-node/x/bridge/`)

No test files exist anywhere under this directory:

- `keeper/msg_server.go` — `MsgDeposit` and `MsgWithdraw` processing
- `keeper/vote_handler.go` — voting logic for deposits and Bridge contract governance messages
- `keeper/relayer.go` — orchestrator polling, deposit processing, and withdrawal confirmation signing

- keeper/abci.go — block-level vote aggregation and state transitions
- keeper/keeper.go — core state management (deposits, pending withdrawals, and confirmed withdrawals)

Impact

Insufficient test coverage increases the likelihood that

- bugs or vulnerabilities remain undetected until production,
- refactors or minor changes unintentionally break existing functionality, and
- critical security assumptions, such as invariant guarantees or permission checks, are violated silently.

Comprehensive testing improves code correctness, auditability, and developer confidence, and it reduces the risk of costly postdeployment fixes.

Recommendations

Create a test suite for the following.

1. The Bridge contract
2. The DEX node
3. End-to-end flows

The Bridge contract

Write tests covering the following at minimum:

- Deposit acceptance and rejection
- The full withdrawal life cycle from `batchedCreateWithdrawals` through `batchedFinalizeWithdrawals`, with assertions that replay is rejected and the dispute period is enforced
- For `checkValidatorSignatures`, verification that the quorum is reached correctly, that signatures supplied out of order fail to reach quorum, and that a single signature from an insufficient-power validator is rejected
- For the emergency-pause flow, verification that the locker voting triggers autopause at threshold, `emergencyUnlock` rotates the validator set atomically, and stale locker votes are cleared on unpause

The DEX node

Tests should cover the following:

- **Input validation** (`msg_server.go`), which should verify that `Withdraw` rejects out-of-range amounts, amounts with excess decimal precision, nonces outside the ± 1 -day window, stale nonces, concurrent pending withdrawals, and insufficient balance. It should also verify that `DepositFinalize` rejects unknown hashes, non-CONFIRMED status, and unelapsed dispute periods.
- **Idempotency and double-spend prevention**, which should verify that a confirmed deposit cannot be reconfirmed, that `DepositFinalize` called twice on the same hash fails on the second call, and that a validator cannot vote twice on the same message hash.
- **Quorum transitions** (`abci.go`), which should verify that votes below `quorumThreshold` leave state unchanged and that reaching the threshold in `EndBlocker` moves deposits to CONFIRMED and withdrawals to CONFIRMED, clears the respective pending collections, and does not reprocess already confirmed entries.
- **Signature correctness** (`keeper.go`), which should verify that `WithdrawConfirm` rejects signatures that recover to an address other than `validator.HotAddress` and rejects confirmations for hashes absent from both `pendingWithdrawals` and `withdrawals`.

End-to-end flows

The most security-critical paths should also be validated end to end, spanning both the orchestrator and the Cosmos keeper. The following should be tested at minimum:

- The **deposit happy path**, which should verify that the EVM deposit event is picked up by the orchestrator and included in `unfinished_deposits`, that validators vote to quorum, that the deposit reaches CONFIRMED, that a user calls `DepositFinalize` after the dispute period, and that tokens are minted.
- The **withdrawal happy path**, which should verify that a user submits `MsgWithdraw` on Cosmos, that tokens are burned, that validators sign via `WithdrawConfirm`, that the withdrawal reaches CONFIRMED, that the orchestrator submits `batchedCreateWithdrawals` on EVM, that the dispute period elapses, and that `batchedFinalizeWithdrawals` transfers USDC to the destination.
- **Blacklisted depositors**, which should verify that a deposit from a blacklisted address is accepted on chain (USDC transferred to bridge) but filtered out of `unfinished_deposits` by the orchestrator and never credited on Cosmos.
- **Subminimum deposits**, which should verify that a deposit below 5 USDC is accepted on chain but silently discarded by the orchestrator, leaving funds locked in the bridge with no Cosmos credit.
- **Bridge contract address rotation with in-flight withdrawals**, which should verify that when the Bridge contract is updated via the governance vote while withdrawals are in `pendingWithdrawals`, pending withdrawal sign data (computed against the old address) should remain valid and complete correctly. Or it should be prohibited to update the Bridge contract address while there are pending withdrawals.
- **Arbitrum contract pause**, which should verify that while `Bridge.sol` is paused, `batchedDepositWithApproval`, `batchedCreateWithdrawals`, and `batchedFinalizeWithdrawals` all revert; the orchestrator should handle the absence of

new events gracefully and resume processing once the contract is unpaused.

Remediation

This issue has been acknowledged by AFX, and a fix was implemented in commit [3d41ff12](#).

3.3. Replayable changeValidatorSet signature can defer rotation finalization indefinitely

Target	Bridge		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

Note: This issue was found by our in-house auditor V12. It has subsequently been validated and verified by our human auditors.

The signed payload built by `changeValidatorSet` contains no nonce, and the function does not call `checkMessageNotUsed` while every other governance function in the contract does. The signature bundle is therefore valid for the same (epoch, hotAddresses, coldAddresses, powers) tuple indefinitely, and anyone who observed the legitimate transaction can resubmit it.

```
function changeValidatorSet(
    ValidatorSetChangeRequest memory newValidatorSet,
    ValidatorSet memory activeHotValidatorSet,
    Signature[] memory signatures
) external whenNotPaused {
    // [...]
    bytes32 data = keccak256(
        abi.encode(
            "changeValidatorSet",
            newValidatorSet.epoch,
            newValidatorSet.hotAddresses,
            newValidatorSet.coldAddresses,
            newValidatorSet.powers
            // no nonce
        )
    );
    bytes32 message = makeMessage(data);
    // no checkMessageNotUsed(message) here either

    changeValidatorSetInner(newValidatorSet, activeHotValidatorSet, signatures,
        message, false);
}
```

Each call overwrites `pendingValidatorSetChange` and assigns `changeTime = block.timestamp`, restarting the dispute timer:

```
function changeValidatorSetInner(
    ValidatorSetChangeRequest memory newValidatorSet,
    ValidatorSet memory activeValidatorSet,
    Signature[] memory signatures,
    bytes32 message,
    bool useColdValidatorSet
) private {
    // [...]
    uint64 changeTime = uint64(block.timestamp);
    pendingValidatorSetChange = PendingValidatorSetChange({
        epoch: newValidatorSet.epoch,
        totalValidatorPower: newTotalValidatorPower,
        changeTime: changeTime,
        changeBlockNumber: getCurBlockNumber(),
        hotValidatorSetHash: newHotValidatorSetHash,
        coldValidatorSetHash: newColdValidatorSetHash,
        nValidators: uint64(newHotValidatorSet.validators.length)
    });
    // [...]
}
```

Impact

While a rotation is pending, anyone who sees the original transaction can replay the bundle once per dispute window to keep `finalizeValidatorSetChange` stuck on `errorCode 3`.

There are several practical effects:

- The bridge continues to operate normally with the current hot validator set — withdrawals and deposits are unaffected.
- Only the *finalization of a queued rotation* is deferred.
- Operators can still rotate via the cold path (`voteEmergencyLock` → `emergencyUnlock`), which uses a nonce plus `checkMessageNotUsed` and cannot be replay deferred.

However, the attacker motive is weak. There is no fund-loss or takeover path — only grieving or extending the response window during an active hot-key compromise (in which case operators escalate to the cold path anyway).

Recommendations

Add a nonce to the signed data and call `checkMessageNotUsed(message)` in `changeValidatorSet`, mirroring the pattern already used in `changeLocker`, `changeFinalizer`,

`changeDisputePeriodSeconds`, `rejectWithdrawals`, `changeBlockDurationMillis`, `changeLockerThreshold`, `emergencyUnlock`, and `onlyUnlock`. The inconsistency itself is the strongest reason to fix this — every sibling governance function has the protection.

Remediation

This issue has been acknowledged by AFX, and a fix was implemented in commit [e7e37890](#).

3.4. Orchestrator is not aware of pausable functionality on Arbitrum

Target	Orchestrator		
Category	Coding Mistakes	Severity	Medium
Likelihood	High	Impact	Medium

Description

Note: This finding was discovered in an out-of-scope orchestrator component. However, the orchestrator is an integral part of the bridging protocol and directly interfaces with the audited code. This finding is documented here because it represents a meaningful correctness and security risk to the system and should be addressed.

The orchestrator is not aware of whether the Bridge smart contract on Arbitrum is paused or not, as it does not make use of any queries to check the pause status of the contract prior to performing bridge-related actions.

Impact

There are two primary impacts:

1. User deposits made on Arbitrum will still be processed by the orchestrator. There is no way to prevent deposits because the orchestrator looks for USDC Transfer events, and USDC transfers cannot be stopped even if the Bridge contract is paused.
2. Withdrawals are still allowed to be made by users on the Cosmos chain but cannot be processed and finalized on Arbitrum while the Bridge contract is paused.

Recommendations

We recommend implementing additional functionality in the orchestrator to handle these two situations:

1. For withdrawals, simply preventing withdrawals to be initiated by a user while the Bridge contract is paused should be sufficient.
2. For deposits, the orchestrator should keep track of all deposits made while the Bridge contract is paused. Later, once the Bridge contract is unpaused, the orchestrator can go back and process all of these deposits.

Remediation

This issue has been acknowledged by AFX.

AFX has provided the following response:

When the bridge is paused, we keep newly submitted withdrawals in the pending voting stage, while still allowing deposits that bypass the bridge.

3.5. Partially voted withdrawals can stay stuck in the pending state indefinitely

Target	Orchestrator		
Category	Coding Mistakes	Severity	High
Likelihood	Low	Impact	Medium

Description

Note: This finding was discovered in an out-of-scope orchestrator component. However, the orchestrator is an integral part of the bridging protocol and directly interfaces with the audited code. This finding is documented here because it represents a meaningful correctness and security risk to the system and should be addressed.

The EndBlocker calls `handlePendingWithdraws()` in each block to handle any pending withdrawals. This function only handles two conditions:

1. When the amount of validator votes for a withdrawal exceeds the quorum threshold, the withdrawal is processed by emitting a `WithdrawConfirmEvent`.
2. When the amount of validator votes for a withdrawal is zero, and the withdrawal has timed out, the withdraw state's `.Pending` field is cleared. This allows the user to submit another withdrawal but leaves the timed-out withdrawal in the `pendingWithdraws` list so it can still be processed by later invocations of `handlePendingWithdraws()`.

However, there is no handling for a withdrawal that has at least one vote but not enough votes to reach the quorum threshold.

Impact

If, for any reason, a withdrawal receives at least one vote but not enough votes to reach the quorum threshold, the withdrawal can remain stuck forever, preventing the user from ever making a new withdrawal until it has been voted on sufficiently. At the time of writing, there exists no way for the user to cancel their withdrawal.

Recommendations

We recommend adding some functionality that allows the user to cancel timed-out withdrawals even if they have at least one validator vote. This functionality would remove the withdrawal from

the `pendingWithdraws` list and refund the user the tokens that were burned to initiate the withdrawal.

Remediation

This issue has been acknowledged by AFX.

AFX has provided the following response:

For withdrawals that have already received at least one vote, we consider them to have a certain degree of legitimacy and prefer to wait for them to gather enough votes. For stuck withdrawals, manual intervention is required.

3.6. Bridge-address updates can strand pending transfers

Target	x/bridge		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Medium

Description

The bridge module derives deposit and withdrawal hashes from the active Bridge contract address. However, a Bridge contract update can be applied while deposits or withdrawals are still in START status. This can split votes for the same transfer across different hashes, because votes collected before the update are keyed by the old bridge address, while votes collected after the update are keyed by the new bridge address.

The EndBlocker applies a pending bridge address as soon as the Bridge contract vote reaches quorum. It does not check whether there are pending deposits, pending withdrawals, or users with a nonempty withdrawState.Pending:

```
func (k Keeper) handlePendingBridgeContract(ctx sdk.Context, quorumThreshold
uint64) {
    // [...]
    if msgSignatures.Power >= uint32(quorumThreshold) {
        k.bridgeContract.Set(sdkCtx, types.BridgeContract{
            Address: pendingBridgeContract.Address,
        })
        k.pendingBridgeContract.Remove(sdkCtx)

        sdkCtx.Logger().Info("bridge-setted", "hash",
            hex.EncodeToString(hash))
    }
}
```

This matters because both deposit and withdrawal hashes include the bridge address as the target:

```
bytes, err = bridgeConnectionIdArguments.Pack(
    common.BytesToAddress(target),
    fixHashBytes,
)
// [...]
return hash1Bytes, nil
```

For withdrawals, the module stores the pending withdrawal under the hash derived from the active bridge address at the time the user initiates the withdrawal. It then records that hash in `withdrawState.Pending` and burns the user's balance:

```
hash, _ := withdraw.GetSignData(bridge.Address)

withdraw.Status = types.MsgStatus_START
withdraw.CreatedAt = uint64(sdkCtx.BlockTime().Unix())
k.pendingWithdraws.Set(ctx, hash, withdraw)
k.msgSignatures.Set(ctx, hash, types.MsgSignatures{})

withdrawState.Nonce = nonce
withdrawState.Pending = hash
err = k.withdrawStates.Set(ctx, user, withdrawState)
// [...]
err = k.dexKeeper.Burn(ctx, user, fixedAmount, 1,
    dexetypes.TransferTag_BRIDGE_BURN, nil)
```

Later, `WithdrawConfirm` recomputes the expected hash using the active bridge address. If the bridge address has changed, the recomputed hash no longer matches the stored pending-withdrawal hash, and the vote is rejected:

```
bridge, err := k.bridgeContract.Get(ctx)
// [...]

expectedHash, signHash := msgWithdraw.GetSignData(bridge.Address)
if !bytes.Equal(expectedHash, hash) {
    return sdkerrors.ErrInvalidRequest.Wrapf("invalid withdraw confirm hash %s", hex.EncodeToString(hash))
}
```

The relayer has the same assumption. It recomputes the withdrawal hash using the active bridge address and skips the withdrawal before requesting a validator signature if the hash differs:

```
messageHash, signHash := withdraw.GetSignData(bridge.Address)
if strings.Compare(hex.EncodeToString(messageHash), item.MessageHash) != 0 {
    continue
}
```

In the time-out branch, if a START withdrawal has zero voting power for long enough, the module only clears `withdrawState.Pending`. It does not refund or remind the burned amount:

```
} else if withdraw.Status == types.MsgStatus_START && msgSignatures.Power
    == 0 {
```

```
blockTime := uint64(sdkCtx.BlockTime().Unix())
if withdraw.CreatedAt > 0 && blockTime
> withdraw.CreatedAt+types.WithdrawPendingTimeout {
    if bytes.Equal(withdrawState.Pending, hash) {
        withdrawState.Pending = []byte{}
        k.withdrawStates.Set(sdkCtx, withdraw.User, withdrawState)
        metricBridgeWithdrawTimeoutUnlocked.Inc()
        sdkCtx.Logger().Info("withdraw-pending-unlocked", "hash",
hex.EncodeToString(hash))
    }
}
```

Deposits are affected in a similar way; Deposit derives the pending-deposit key from the active bridge address whenever a vote is handled:

```
bridge, err := k.bridgeContract.Get(ctx)
// [...]
hash, _ := deposit.GetSignData(bridge.Address)

if has, _ := k.pendingDeposits.Has(ctx, hash); !has {
    if has, _ := k.deposits.Has(ctx, hash); !has {
        deposit.Status = types.MsgStatus_START
        k.pendingDeposits.Set(ctx, hash, deposit)
    }
}
// [...]
k.msgSignatures.Set(ctx, hash, msgSignatures)
```

If some validators vote for a deposit before the bridge-address update and the remaining validators vote after it, their voting power is accumulated under different hashes. The deposit may then fail to reach quorum until validators coordinate a full revote under one bridge address.

Impact

Users with in-flight withdrawals can have their burned tokens stranded if the Bridge contract address changes before the withdrawal reaches quorum. Subsequent votes may fail the hash check or be skipped by the relayer, and the time-out path only lets the user submit another withdrawal without restoring the burned balance.

Users with in-flight deposits can also be delayed or blocked from receiving credit on the Cosmos side when votes are split between old and new hashes.

Recommendations

Block bridge-address updates while `pendingWithdraws` or `pendingDeposits` contain `START` transfers, unless the update also performs an explicit migration or refund.

Remediation

This issue has been acknowledged by AFX.

AFX has provided the following response:

The likelihood of changing the bridge address is extremely low. If required, orchestrators are paused first, and we ensure that all bridge tickets have reached a finalized state before proceeding.

3.7. Attacker can deposit a victim's preapproved USDC to the Cosmos chain without consent

Target	Bridge		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

The `batchedDepositWithApproval` function is callable by any address and accepts arbitrary user values. Any user who has approved the Bridge contract can have their entire USDC allowance forcibly deposited to the Cosmos chain by a third party, without any action or consent from the user.

The function places no restriction on the caller and does not verify that `msg.sender` is the depositing user:

```
function batchedDepositWithApproval(
    DepositWithApproval[] memory deposits
) external nonReentrant whenNotPaused {
    uint64 end = uint64(deposits.length);
    for (uint64 idx; idx < end; idx++) {
        depositWithApproval(deposits[idx].user, deposits[idx].usd);
    }
}

function depositWithApproval(address user, uint64 usd) private {
    address spender = address(this);
    try usdcToken.transferFrom(user, spender, usd) returns (bool success) {
        ...
    }
}
```

For deposits at or above the 5 USDC orchestrator minimum, the relayer processes them normally and credits the user address on the Cosmos chain. Consider the following attack scenario:

1. The user Alice approves the Bridge for a large allowance (e.g., `MAX_UINT64`) in preparation for a future deposit or as a one-time unlimited approval.
2. An attacker calls `batchedDepositWithApproval([user: Alice, usd: Alice's full balance])`.
3. Then `transferFrom(Alice, bridge, balance)` succeeds. Alice's EVM USDC balance is drained to zero.

4. The orchestrator sees the deposit (≥ 5 USDC), the relayer processes it, and Alice's Cosmos account is credited.
5. Alice has lost all EVM access to her USDC. To recover it, she must initiate and finalize a withdrawal through the bridge's Cosmos-side withdrawal flow and wait out the full dispute period.

The attack requires no privileged access. Approvals are publicly visible on chain, making any holder of a large bridge approval a target.

Impact

A victim's entire preapproved USDC balance is moved from their EVM wallet to the bridge contract without consent. Although the victim receives an equivalent Cosmos credit, the forced bridging constitutes an unauthorized transfer that

- removes immediate EVM access to the funds;
- forces the victim to go through the bridge's withdrawal process (dispute period, validator signatures, and finalizer call) to retrieve their funds; and
- leaves victims without Cosmos accounts or awareness effectively unable to recover without operator assistance.

This finding is also the prerequisite for the Critical dust-drain issue (see Finding [3.1.7](#)), where the same permissionless path is used with subminimum amounts that are discarded by the orchestrator, resulting in permanent fund loss.

Recommendations

Restrict `batchedDepositWithApproval` so that only the depositing user may trigger a deposit on their behalf. The simplest fix is to enforce `msg.sender == deposit.user` for each entry:

```
function batchedDepositWithApproval(
    DepositWithApproval[] memory deposits
) external nonReentrant whenNotPaused {
    uint64 end = uint64(deposits.length);
    for (uint64 idx; idx < end; idx++) {
        require(deposits[idx].user == msg.sender, "Unauthorized depositor");
        depositWithApproval(deposits[idx].user, deposits[idx].usd);
    }
}
```

Remediation

This issue has been acknowledged by AFX.

AFX has provided the following response:

Our service only supports users completing a gasless approval via permit, and then calling `batchedDepositWithApproval` to complete the USDC transfer. For every permit, we believe it is the amount that the user want to deposit.

3.8. Validator rotation can double-count bridge votes

Target	x/bridge		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Medium

Description

Note: This issue was found by our in-house auditor V12. It has subsequently been validated and verified by our human auditors.

The bridge module stores vote state as a cached `msgSignatures.Power` total and deduplicates signers by `validator.HotAddress`. The stored vote state is not bound to a validator-set version, and end-block processing does not recompute signer membership or power against a single active validator set. This allows votes from different validator-set versions to be mixed for the same bridge message.

Votes are recorded by hot address, and the validator's power is added to the cached total:

```
if msgSignatures.HasSigned(validator.HotAddress) {
    return sdkerrors.ErrInvalidRequest.Wrapf("validator %s has signed this message", hex.EncodeToString(validator.HotAddress))
}

msgSignatures.Signatures = append(msgSignatures.Signatures,
    types.ValidatorSignature{
        Validator: validator.HotAddress,
    })
msgSignatures.Power = msgSignatures.Power + validator.Power
k.msgSignatures.Set(ctx, hash, msgSignatures)
```

And `HasSigned` only compares the stored signer bytes to the validator's current hot address:

```
func (m *MsgSignatures) HasSigned(addr sdk.AccAddress) bool {
    for _, item := range m.Signatures {
        if bytes.Equal(item.Validator, addr) {
            return true
        }
    }
    return false
}
```



```
}
```

The EndBlocker later trusts the cached power total and compares it to the current quorum threshold:

```
validatorSet, err := k.vericonKeeper.GetValidatorSet(ctx)
// [...]
k.handlePendingBridgeContract(sdkCtx, validatorSet.QuorumThreshold)
k.handlePendingDeposits(sdkCtx, validatorSet.QuorumThreshold)
k.handlePendingWithdrawals(sdkCtx, validatorSet.QuorumThreshold)

// [...]

if deposit.Status == types.MsgStatus_START && msgSignatures.Power
    >= uint32(quorumThreshold) {
    deposit.Status = types.MsgStatus_CONFIRMED
    // [...]
}
```

If a validator votes before rotating its hot key, the stored signature remains keyed by the old hot address. After rotation, the same validator can be passed to the bridge module with a new `validator.HotAddress`, so `HasSigned` does not detect the prior vote and the validator's power can be added again. More generally, pending vote totals can contain stale votes from an old validator set while being evaluated against the current set's quorum.

The Solidity bridge avoids this class of issue by checking signatures against a supplied validator-set hash that includes validators, powers, and epoch:

```
require(
    makeValidatorSetHash(activeValidatorSet) == validatorSetHash,
    "Supplied active validators and powers do not match the active checkpoint"
);
```

The Cosmos bridge module does not have an equivalent epoch binding for `msgSignatures`.

Impact

Stale or double-counted voting power can push a pending bridge action over quorum even though one validator-set version did not approve it. The deposit path is the highest-impact case, because a fabricated pending deposit that reaches quorum can later be finalized through `DepositFinalize` and mint unbacked DEX balance.

Recommendations

Bind every `msgSignatures` entry to a validator-set epoch or hash, and reject votes for a message once the active validator set changes. When checking quorum, recompute signed power from immutable validator identities in the same validator set instead of trusting a cached total keyed only by hot address.

Remediation

This issue has been acknowledged by AFX.

AFX has provided the following response:

- Under normal circumstances, `votingUserDeposits` prevents duplicate voting on the same deposit.
- During validator rotation, orchestrators are paused first, and we ensure that all bridge tickets have reached a finalized state before proceeding.

3.9. Relayer vote caches grow without bound

Target	x/bridge		
Category	Coding Mistakes	Severity	Medium
Likelihood	High	Impact	Medium

Description

Note: This issue was found by our in-house auditor V12. It has subsequently been validated and verified by our human auditors.

The bridge relayer keeps pending deposit and withdrawal votes in global `sync.Map` caches. New deposits and withdrawals are inserted into `votingUserDeposits` and `votingUserWithdraws`, but there is no deletion path after a vote is emitted or after the bridge operation is finalized.

The caches are process-global state:

```
var (
    // [...]
    votingUserDeposits sync.Map
    votingUserWithdraws sync.Map
    votingBridgeContractSet *VotingBridgeContractSet
)
```

The relayer workers append new deposit and withdrawal work whenever they observe a new key:

```
_, has := votingUserDeposits.Load(key)
if has {
    continue
}
// [...]
votingUserDeposits.Store(key, &VotingUserDeposit{
    Data: data,
    Voted: false,
})
// [...]

_, has := votingUserWithdraws.Load(item.MessageHash)
if has {
```

```
        continue
    }
    // [...]
    votingUserWithdraws.Store(item.MessageHash, &VotingUserWithdraw{
        Data: data,
        Voted: false,
    })
}
```

And GetPendingVotes scans both maps and only marks entries as voted. It does not evict entries after returning them:

```
votingUserDeposits.Range(func(key, value any) bool {
    item := value.(*VotingUserDeposit)
    if item.Voted == true {
        return true
    }
    pendingVotes = append(pendingVotes, &votes.PendingVote{
        VoteType: types.BridgeVote_UserDeposit,
        VoteData: item.Data,
    })
    item.Voted = true
    return true
})

votingUserWithdraws.Range(func(key, value any) bool {
    item := value.(*VotingUserWithdraw)
    if item.Voted == true {
        return true
    }
    pendingVotes = append(pendingVotes, &votes.PendingVote{
        VoteType: types.BridgeVote_UserWithdraw,
        VoteData: item.Data,
    })
    item.Voted = true
    return true
})
```

Because deposits and withdrawals are user-triggerable, an attacker can keep creating fresh bridge activity and force validator processes to retain an ever-growing historical backlog.

Impact

Validator processes running the relayer workers can accumulate unbounded memory and CPU overhead. As the maps grow, GetPendingVotes must rescan more historical entries, which can

slow vote production and eventually cause nodes to stall or crash until operators restart or patch them.

Recommendations

Delete entries from `votingUserDeposits` and `votingUserWithdraws` after they are emitted, finalized, or otherwise no longer needed.

Remediation

This issue has been acknowledged by AFX.

AFX has provided the following response:

We have confirmed this issue. As bridge operations continue to accumulate, the impact of this issue will become more noticeable over time. Given our relatively frequent system releases, we believe this issue will not have a material impact on our business in the short term. We plan to fix it shortly.

3.10. Multiple issues related to the Bridge contract address update functionality

Target	DEX module		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Low

Description

The DEX module allows validators to vote on a new Bridge contract address. On the very first vote, the `pendingBridgeContract` state variable is set to the address that the vote was for. All subsequent votes must then be for this same address. There is no functionality that allows changing the `pendingBridgeContract` address at this point until it is finalized.

Impact

A single malicious validator can set the `pendingBridgeContract` address to something completely malicious and thus brick this flow completely. The only fix here would be to hard fork the Cosmos chain and implement some functionality that allows the `pendingBridgeContract` address to be cleared.

Recommendations

We have two recommendations here:

1. Disallow setting `pendingBridgeContract` through a single validator vote. All votes should be accumulated, and any address that reaches quorum should be allowed to become the new Bridge contract.
2. Implement a privileged message (potentially via validator voting) that allows clearing the `pendingBridgeContract` address.

Remediation

This issue has been acknowledged by AFX.

AFX has provided the following response:

We plan to remove the functionality that allows changing the bridge contract through validator voting.

3.11. Nil validator in bridge vote handling can panic

Target	x/bridge		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The bridge vote handler does not reject a nil validator before routing the vote. If `HandleVote` is ever called with a nil validator, `convertValidator` returns nil, and that nil value is passed into the deposit, withdrawal, or Bridge contract vote handler:

```
func (k Keeper) HandleVote(ctx context.Context, validator
*vericontypes.Validator, totalPower uint32, voteType string, voteData
[]byte) error {
    sdkCtx := sdk.UnwrapSDKContext(ctx)

    bridgeValidator := convertValidator(validator)

    switch voteType {
    case bridgetypes.BridgeVote_UserDeposit:
        return k.HandleUserDepositVoteMsg(sdkCtx, bridgeValidator, voteData)
    case bridgetypes.BridgeVote_UserWithdraw:
        return k.HandleUserWithdrawVoteMsg(sdkCtx, bridgeValidator, voteData)
    case bridgetypes.BridgeVote_BridgeContract:
        return k.HandleBridgeContractVoteMsg(sdkCtx, bridgeValidator,
voteData)
    default:
        return sdkerrors.ErrInvalidRequest.Wrapf("unsupported bridge vote
type: %s", voteType)
    }
}

func convertValidator(v *vericontypes.Validator) *types.Validator {
    if v == nil {
        return nil
    }
    return &types.Validator{
        Address:      v.Address,
        HotAddress:   pubkeyToAddress(v.HotAddress),
        ColdAddress:  pubkeyToAddress(v.ColdAddress),
        Power:        v.Power,
```

```
}  
}
```

Each routed path then dereferences `validator.HotAddress` without validating `validator`. The bridge-contract path dereferences it immediately, and the deposit and withdrawal paths have the same pattern:

```
func (k Keeper) BridgeContract(ctx context.Context, validator  
    *types.Validator, contractAddr sdk.AccAddress) error {  
    sdkCtx := sdk.UnwrapSDKContext(ctx)  
    sdkCtx.Logger().Debug("bridge_contract", "validator",  
        sdk.AccAddress(validator.HotAddress), "address",  
        sdk.AccAddress(contractAddr))  
  
    // [...]  
    if msgSignatures.HasSigned(validator.HotAddress) {  
        return sdkerrors.ErrInvalidRequest.Wrapf("validator %s has signed this  
message", hex.EncodeToString(validator.HotAddress))  
    }  
}  
  
func (k Keeper) Deposit(ctx context.Context, validator *types.Validator, user  
    sdk.AccAddress, amount uint64, deadline uint64, txHash []byte, index  
    uint64) error {  
    sdkCtx := sdk.UnwrapSDKContext(ctx)  
    sdkCtx.Logger().Info("deposit", "validator",  
        sdk.AccAddress(validator.HotAddress), "user", sdk.AccAddress(user),  
        "amount", amount, "deadline", deadline)  
    // [...]  
}  
  
func (k Keeper) WithdrawConfirm(ctx context.Context, validator  
    *types.Validator, hash []byte, signature []byte) error {  
    // [...]  
    if msgSignatures.HasSigned(validator.HotAddress) {  
        return sdkerrors.ErrInvalidRequest.Wrapf("validator %s has signed this  
message", hex.EncodeToString(validator.HotAddress))  
    }  
    // [...]  
}
```

The audited bridge code does not show the full caller path that supplies validators to `HandleVote`, so we could not confirm that an untrusted caller can cause `validator` to be `nil`. However, any reachable path that passes `nil` into `HandleVote` would trigger a panic instead of returning an error.

Impact

If `HandleVote` is called in `EndBlocker/BeginBlocker` with a nil validator, it will trigger a panic instead of returning an error.

Recommendations

Add an explicit nil check in `HandleVote` before calling `convertValidator`.

Remediation

This issue has been acknowledged by AFX, and a fix was implemented in commit [bf4b083f](#).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Missing access to the full codebase

During the audit, we noticed that both the orchestrator and the DEX module integrate with external modules and components whose implementations were not provided to us. As a result, we treated those external components as black boxes and made several assumptions about their behavior. Because of this limitation, we may have missed issues that could have been identified if access to those external components had been available.

4.2. Signature checks missing from two bridge vote-related messages

During the audit, we noticed that the `HandleUserWithdrawVoteMsg` handler flow calls `WithdrawConfirm()`, which has some signature checks:

```
pub, err := ethcrypto.SigToPub(signHash, signature)
if err != nil {
    sdkCtx.Logger().Error("invalid withdraw signature", "error", err)
    return sdkerrors.ErrInvalidRequest.Wrapf("invalid withdraw signature")
}
recovered := ethcrypto.PubkeyToAddress(*pub)
if !bytes.Equal(recovered.Bytes(), validator.HotAddress) {
    return sdkerrors.ErrInvalidRequest.Wrapf("signature does not match
validator hot address, validator %s, recovered %s",
hex.EncodeToString(validator.HotAddress), recovered.Hex())
}
```

However, these checks are missing in the `HandleUserDepositVoteMsg` and `HandleBridgeContractVoteMsg` flows. Since we did not have access to the code that actually calls into these vote message flows, we are not certain that this is an issue. However, we raise it for discussion so that it could be addressed if there is any impact.

4.3. Possible quorum-check mismatch between the bridge module and Bridge contract

The bridge module's `EndBlocker` accepts withdrawals when `msgSignatures.Power >= quorumThreshold` (with `quorumThreshold` supplied by the vericon module), while the Bridge contract's `checkValidatorSignatures` requires `3 * cumulativePower > 2 * totalValidatorPower`. Both are intended to enforce the same 2/3 quorum, but they use different operators (`>=` vs strict `>`) and pull the threshold from different sources: `quorumThreshold` is supplied by the vericon module, while the contract derives its bound from its own `totalValidatorPower`. If vericon computes `quorumThreshold` as `floor(2 * totalPower / 3)` (or any formula that omits the +1), there is a power level at which the bridge module would confirm a withdrawal that the Bridge contract then rejects, leaving user funds burned with no on-chain payout.

4.4. Bridge contract hashing can panic in the EndBlocker

The bridge `EndBlocker` calls `GetSignData` while processing a pending Bridge contract update. This helper panics on internal errors instead of returning early, so any reachable failure in this path can become an unrecovered panic during end-block processing.

```
func (k Keeper) handlePendingBridgeContract(ctx sdk.Context, quorumThreshold
uint64) {
    // [...]
    pendingBridgeContract, err := k.pendingBridgeContract.Get(sdkCtx)
    if err != nil {
        sdkCtx.Logger().Error("handlePendingBridgeContract", "error", err)
        return
    }

    hash,
    _ := pendingBridgeContract.GetSignData(pendingBridgeContract.Address)
    // [...]
}
```

Here, `BridgeContract.GetSignData` panics on errors returned from `ConnectionId`, `HashStruct`, and `TypedDataAndHash`:

```
func (m *BridgeContract) GetSignData(target []byte) ([]byte, []byte) {
    connectionId, err := m.ConnectionId(target)
    if err != nil {
        panic(err)
    }
    // [...]
```

```
hash1Bytes, err := typedData.HashStruct(typedData.PrimaryType,
typedData.Message)
if err != nil {
    panic(err)
}

hash2Bytes, _, err := apitypes.TypedDataAndHash(typedData)
if err != nil {
    panic(err)
}

return hash1Bytes, hash2Bytes
}
```

If one of these helpers fails while `pendingBridgeContract` is set, end-block processing could panic instead of returning an error, which could lead to chain halt as panics in the `EndBlocker` are not recovered. The `EndBlocker` flow should handle hashing failures gracefully instead of relying on panic. We recommend handling these errors and returning early instead of panicking.

Remediation

This issue has been acknowledged by AFX, and a fix was implemented in commit [80096e68](#).

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: Orchestrator

Description

The orchestrator is the off-chain validator process that completes the bridge between Arbitrum and the Cosmos chain. The x/bridge module on the Cosmos chain communicates with Arbitrum using this orchestrator. Each validator runs its own orchestrator, which watches both chains, converts observed events into the bridge module's expected messages, and signs or submits the next step in the flow.

The orchestrator is responsible for the following:

- Watching Arbitrum for USDC Transfer events into the Bridge contract
- Watching the Cosmos chain for the following events: `WithdrawEvent`, `WithdrawConfirmEvent`, `DepositConfirmEvent`, and `DepositFinalizeEvent`
- Relaying deposits from Arbitrum to the Cosmos chain by submitting validator votes
- Relaying withdrawals from the Cosmos chain to Arbitrum by collecting the validator set and signatures then calling `batchedCreateWithdrawals()` and later `batchedFinalizeWithdrawals()`
- Cross-checking Arbitrum RPC results with Etherscan and continuously monitoring bridge safety conditions via the emergency-lock monitor

The above responsibilities are split into a few long-running loops:

- Poll Arbitrum's latest height, and, once enough confirmations have passed, fetch bridge-related logs.
- Poll the Cosmos chain's latest height, and, once enough confirmations have passed, fetch bridge events from transactions.
- Process the events into the next bridge action.
- Periodically update metrics, and, if enabled, run the emergency-lock monitor.

Invariants

- Only sufficiently confirmed chain data should be acted on. Both Arbitrum and the Cosmos chain polling wait for appropriate confirmation depths before acting on any events.
- A single orchestrator instance should not send conflicting Arbitrum transactions with the same nonce.

- Reprocessing old blocks should be always result in the same outcome.
- Duplicate withdrawal creation/finalization should not lead to double-spending/execution on Arbitrum. Before sending, the orchestrator checks `usedMessages` and `finalizedWithdrawals`. Even if race conditions cause these checks to pass in situations where they should fail, the Bridge contract will prevent double-spending/execution via message replay checks.
- Restart recovery should preserve the bridge progress. The orchestrator reloads `state.json`, repairs partially written entries with `recoverIncompleteState()`, and resumes from persisted heights unless operators explicitly reset or replay ranges.
- Blacklisted users should not be able to withdraw or deposit via the bridge.
- The emergency lock should trip when key safety assumptions break. The monitor checks validator set consistency, minimum dispute period, and oversized withdrawals then enqueues `voteEmergencyLock()` if a violation is observed.

Test coverage

Cases not covered

- The end-to-end deposit relay flow — Arbitrum deposit detection → vote submission → Cosmos chain confirmation/finalization
- The end-to-end withdraw relay flow — Cosmos chain withdraw detection → signature aggregation → `batchedCreateWithdrawals()` → timer-driven `batchedFinalizeWithdrawals`
- Replay/reset semantics and crash recovery via `state.json`, including `recoverIncompleteState`
- Multi-orchestrator races and idempotency guarantees around `usedMessages`, `finalizedWithdrawals`, and duplicate event processing
- Emergency-lock monitor behavior
- Blacklist handling, minimum/maximum amount thresholds, and retry logic

Attack surface

The orchestrator sits on a large trust boundary because it consumes external data from both chains and can submit privileged bridge actions. The main attacker-controlled inputs are as follows:

- **Arbitrum events.** Deposits, created withdrawals, finalized withdrawals, and contract reads all come from configured RPC endpoints. Optional Etherscan verification reduces but does not fully eliminate any risks.
- **DEX chain events.** Withdrawals, withdraw confirmations, and deposit confirmations are taken from chain events and gRPC query responses. The orchestrator assumes those endpoints are honest and available.
- **Bridge activity.** Users choose deposit amounts, withdraw destination, nonce, and timing. The orchestrator reacts to whatever valid bridge activity happens on chain.

- **Operator HTTP endpoints.** The endpoints `/replay/*`, `/reset/*`, and `/emergency_lock/*` are powerful and unauthenticated in-process endpoints. The default bind is localhost, so there should be no issues, but incorrectly exposing these endpoints can allow for attacks.
- **Paused contract recovery path.** When the Arbitrum contract is paused, Cosmos chain withdrawals can still be accepted and user balances can still be burned, but the orchestrator's Arbitrum task is dropped on failure and is not automatically replayed later. Recovery depends on operators noticing the condition and using the right replay endpoint after unpauses.

5.2. Component: Deposits

Description

On the Arbitrum side, USDC enters the bridge through three paths: `batchedDepositWithApproval`, `batchedDepositWithPermit`, or a plain transfer of USDC directly to the bridge address. The orchestrator does not distinguish between these paths. It filters on any USDC Transfer event whose to address is the bridge and credits the from address on the Cosmos chain. The contract-side `whenNotPaused` and `nonReentrant` guards therefore only apply to the two batched entry points; a direct ERC-20 transfer bypasses them entirely. The permit path is intentionally permissionless (the permit signature is the user's authorization), but the approval path is also callable by any third party for any preapproved user, which is the bug described in Finding [3.7](#). Per-entry failures in the batched paths are demoted to `FailedPermitDeposit` / `FailedApprovalDeposit` events so a single bad entry does not revert the batch.

After a deposit is observed on the Bridge smart contract, validators vote on the Cosmos side through `HandleVote`. During voting, deposits are stored in the `pendingDeposits` map, keyed by the deposit hash. The corresponding `msgSignatures` record tracks which validators have voted and the cumulative validator `Power` for that deposit.

The `EndBlocker` calls `handlePendingDeposits`, which walks through all pending deposits. If a deposit is still in `types.MsgStatus_START` and its cumulative voting power is at least `quorumThreshold`, the deposit is marked as `types.MsgStatus_CONFIRMED`, its `ConfirmedAt` timestamp is set, and it is removed from `pendingDeposits`.

After `DepositDisputePeriod` has elapsed from `ConfirmedAt`, the deposit can be finalized through `DepositFinalize`. Finalization mints the corresponding tokens to the depositor on the Cosmos chain and sets the deposit status to `types.MsgStatus_FINALIZED`.

Invariants

- A validator should only be able to vote once per deposit.
- A deposit should only be confirmed when cumulative validator power is at least `quorumThreshold`.
- Confirmed deposits should be removed from `pendingDeposits`.
- Deposits should only be finalized if they are confirmed and `DepositDisputePeriod` has

elapsed.

- A finalized deposit should not be finalized again or mint tokens more than once.
- The deposit hash should uniquely bind the depositor, amount, deadline, Arbitrum transaction hash, transaction index, and Bridge contract address.
- Deposits should be rejected when the Arbitrum contract is paused.
- The Cosmos-minted amount should be equal the USDC amount actually transferred on Arbitrum.

Test coverage

Cases not covered

- The end-to-end deposit flow
- Unit tests for each deposit function and deposit status transition
- Tests for deposits where validators do not vote
- Tests for duplicate validator votes on the same deposit
- Tests for deposits where cumulative validator power is below `quorumThreshold`
- Tests that finalized deposits cannot be finalized again

Attack surface

The Arbitrum entry is permissionless. The approval path lets any third party force-deposit a victim's outstanding allowance (see Finding [3.7](#), ↗); a plain `usdc.transfer` to the bridge is also credited, since detection is event-based and bypasses the contract's `whenNotPaused` guard. Subminimum amounts are silently absorbed (see Finding [3.1](#), ↗).

After deposits reach the Cosmos chain, validators vote on them through `HandleVote`. The main attack surface is the vote path and the deposit data supplied to that path.

The `Deposit` function itself does not verify a validator signature over the deposit. It relies on `HandleVote` / `vericon` to authenticate the validator and provide the correct validator identity. If that upstream authentication or validator conversion is incorrect, malformed or unauthorized votes may be accepted. This issue is discussed in detail in Finding [4.2](#), ↗.

The Cosmos chain also does not independently verify Arbitrum consensus or prove that the deposit occurred on chain. Deposit validity is ultimately accepted through the validator voting process, so compromised or colluding validators can cause unbacked deposits to be confirmed and finalized.

5.3. Component: Withdrawals

Description

Users start a withdrawal by submitting `MsgWithdraw`, which calls `Withdraw` in `msg_server.go`. The message validates the destination address, amount precision, minimum and maximum withdrawal limits, and checks that the user-provided nonce is within one day of the current block time.

The keeper then checks the user's `WithdrawState`. A user must provide a nonce greater than their previous nonce and must not already have a pending withdrawal. The withdrawal amount plus the configured withdrawal fee must be available in the user's balance.

After validation, the keeper creates a withdrawal object, computes its hash using the configured Bridge contract address, stores it in `pendingWithdraws`, initializes `msgSignatures` for the hash, updates `withdrawStates[user].Pending`, burns the withdrawal amount, and transfers the fee to the fee collector if one is configured.

Validators confirm the withdrawal through `WithdrawConfirm`. Each validator provides a signature over the withdrawal message. The keeper recovers the signer from the signature and verifies that it matches the validator's hot address. Each validator should only be counted once, and their power is accumulated in `msgSignatures`.

The `EndBlocker` calls `handlePendingWithdraws`, which walks through all pending withdrawals. If a withdrawal is still in `types.MsgStatus_START` and its cumulative voting power is at least `quorumThreshold`, the withdrawal is marked as `types.MsgStatus_CONFIRMED`, moved to `withdraws`, and removed from `pendingWithdraws`, and the user's pending withdrawal state is cleared.

After Cosmos confirmation, the orchestrator collects validator signatures and submits them to Arbitrum via `batchedCreateWithdrawals`. The function is permissionless; any caller can submit a list of withdrawals together with the active hot validator set. For each entry, the contract reconstructs the message hash from `(user, destination, usd, nonce)`, rejects it if it already exists in `createdWithdrawals` or appears in `withdrawalsRejected`, and runs `checkValidatorSignatures` to verify if cumulative hot-validator power is greater than 2/3. Signatures must be supplied in canonical validator-set order, since the matching cursor only advances forward. It then stores a `Withdrawal` record in `createdWithdrawals[message]`, stamped with the current block timestamp and the current L2 block number. Both stamps start the dual dispute clocks that the gate finalizes.

After the dispute period elapses, an address in the `finalizers` set calls `batchedFinalizeWithdrawals`. The contract checks both the timestamp and L2 block-number clocks before marking `finalizedWithdrawals[message] = true` and paying out via `safeTransfer`. Cold validators can veto any pending or future message via `rejectWithdrawals`, which is the emergency brake on a compromised hot key.

If a pending withdrawal receives no validator votes and `WithdrawPendingTimeout` has elapsed, `handlePendingWithdraws` clears the user's pending withdrawal state so the user can submit another withdrawal. This time-out path does not refund the burned withdrawal amount or fee.

Invariants

- A user should not be able to create more than one pending withdrawal at a time.
- Withdrawal nonces should be strictly increasing per user.
- The withdrawal hash should uniquely bind the user, destination, amount, nonce, and Bridge contract address.
- A validator should only be able to vote once per withdrawal.
- A validator signature should only count if it recovers to that validator's hot address.
- A withdrawal should only be confirmed when cumulative validator power is at least `quorumThreshold`.
- On Arbitrum, a withdrawal should only be created with a hot-validator signature set whose cumulative power is strictly greater than $2/3$ of `totalValidatorPower`.
- Confirmed withdrawals should be removed from `pendingWithdraws`.
- A user's pending withdrawal state should be cleared once the withdrawal is confirmed or safely canceled.
- Only addresses in `finalizers` should call `batchedFinalizeWithdrawals`.

Test coverage

Cases not covered

- The end-to-end withdrawal flow
- Unit tests for each withdrawal status transition
- Tests for invalid amount precision, nonce, minimum amount, and maximum amount
- Tests for users attempting to create a second withdrawal while one is pending
- Tests for duplicate validator votes on the same withdrawal
- Tests for invalid or malformed validator signatures
- Tests for withdrawals where cumulative validator power is below `quorumThreshold`
- Tests for withdrawals that receive no validator votes and pass `WithdrawPendingTimeout`
- Tests for withdrawals that receive some validator votes but never reach quorum

Attack surface

The withdrawal flow has a significant liveness and fund-safety attack surface because user funds are burned before the withdrawal reaches validator quorum.

The main attack surface is `MsgWithdraw`, `WithdrawConfirm`, and the pending-withdrawal state machine. User-controlled inputs include the destination address, amount, and nonce. Validator-controlled inputs include the withdrawal confirmation signature and the validator identity supplied through the vote path.

The `WithdrawConfirm` function verifies that the recovered signer matches the validator's hot address, but it still relies on `HandleVote` / `vericon` to authenticate the validator and provide the correct validator object and power. If that upstream path is incorrect, malformed votes or validator

identity issues may affect withdrawal confirmation.

A key liveness risk exists when a withdrawal is burned but does not reach quorum. If `msgSignatures.Power == 0` and `WithdrawPendingTimeout` passes, `handlePendingWithdraws` only clears `withdrawStates[user].Pending`; it does not refund the burned withdrawal amount or the fee. The user can submit a new withdrawal, but the original burned funds remain lost unless validators later confirm the original withdrawal.

An even stronger stuck-funds case exists when at least one validator votes but the withdrawal never reaches quorum. In that case, `msgSignatures.Power != 0`, so the zero-vote time-out path does not run. The withdrawal can remain pending indefinitely with the user's funds already burned. This issue is discussed more in [Finding 3.5](#). ↗

On the Arbitrum side, the caller-supplied inputs to `batchedCreateWithdrawals` are the `hotValidatorSet` (rejected if its hash does not match `hotValidatorSetHash`) and the signatures array (silently fails to reach quorum if out of canonical order). A created withdrawal sits in `createdWithdrawals` indefinitely until finalized or vetoed — there is no on-chain expiry. The most consequential liveness gap is a paused Arbitrum contract; Cosmos keeps accepting and burning withdrawals while Arbitrum cannot create or finalize, and the orchestrator has no pause awareness (see [Finding 3.4](#), ↗). Cold-key compromise enables `rejectWithdrawals` to veto arbitrary withdrawals; combined with the Cosmos-side burn before quorum, a vetoed withdrawal leaves user funds burned with no on-chain refund path on either side.

5.4. Component: Bridge contract address change

Description

The bridge module stores the configured Bridge contract address in `bridgeContract`. This address is used when computing the signed message hashes for deposits and withdrawals, so changing it affects how future bridge messages are authenticated and identified.

Validators vote on the Bridge contract address through `HandleVote` using the `BridgeVote_BridgeContract` vote type, and `HandleBridgeContractVoteMsg` validates its address format and calls `BridgeContract` in `keeper.go`.

The first vote for a Bridge contract address creates `pendingBridgeContract` if it does not already exist. Validator votes are tracked in `msgSignatures`, and each validator's power is accumulated for the pending Bridge contract address.

The `EndBlocker` calls `handlePendingBridgeContract`. If the pending Bridge contract address has accumulated voting power greater than or equal to `quorumThreshold`, the keeper updates `bridgeContract` to the pending address and removes `pendingBridgeContract`.

Invariants

- The Bridge contract address should only be changed after validator quorum is reached.
- A validator should only be able to vote once for the same Bridge contract address

proposal.

- A pending Bridge contract proposal should be removable, replaceable, or rejectable through a safe mechanism if it is incorrect.
- A single validator should not be able to permanently block future Bridge contract address changes.
- Once quorum is reached, `bridgeContract` should be updated exactly once and `pendingBridgeContract` should be cleared.
- Future deposit and withdrawal hashes should use the latest finalized Bridge contract address.

Test coverage

Cases not covered

- The end-to-end flow of changing the Bridge contract address
- Unit test for voting
- `EndBlocker` tests to validate that the Bridge contract address is updated correctly and the pending Bridge contract is cleared

Attack surface

The main attack surface is the Bridge contract address vote path.

The Bridge contract address change flow is especially sensitive because the configured address is part of deposit and withdrawal hash generation. If the wrong address is finalized, future bridge messages may be bound to the wrong contract address, causing bridge liveness failures or incorrect message validation.

A key liveness risk exists in the pending proposal state. The first validator vote can create `pendingBridgeContract`. Once this pending address exists, later votes for a different address do not replace or clear it. There is no time-out, rejection, or cancellation path for an incorrect pending Bridge contract address. This is discussed more in Finding [3.10](#). ↗

6. Assessment Results

During our assessment on the scoped AFX Bridge contracts, we discovered 11 findings. Two critical issues were found. One was of high impact, six were of medium impact, one was of low impact, and the remaining finding was informational in nature.

Based on the scope of findings identified during this engagement, we recommend a comprehensive reassessment of the codebase following remediation efforts. This is particularly important given that the audit covered only a subset of the components that together form the bridging protocol, and the absence of test coverage across all security-critical paths limits both our ability to verify correctness and AFX's ability to safely maintain the system going forward. Additionally, a key factor necessitating a reaudit is that we did not have the ability to run or interact with the system in a live or local environment. This significantly restricted our ability to validate functionality, explore edge cases, and assess system behavior beyond static review.

A follow-up assessment would provide greater confidence in the overall security posture of the protocol and help ensure that remediation efforts have not introduced new issues.

To maximize the value of a follow-up engagement, we recommend first addressing the following items:

- **Provide the source code of all external dependencies for context.** As noted in Discussion section [4.1](#), the bridge module's deposit and withdrawal confirmation logic relies on the vericon module for validator identity preverification and quorum threshold computation and on the DEX module for token minting, burning, and balance queries. Neither was provided during this engagement, requiring us to treat them as black boxes. For a follow-up assessment, AFX should share the source code of all such dependencies so reviewers can verify the security assumptions that cross module boundaries — even if those modules are not themselves formally in scope.
- **Bring the orchestrator into scope.** The orchestrator was not a primary audit target for this engagement, yet it is the component that bridges EVM events to the Cosmos chain, filters deposits by minimum amount, and drives the withdrawal signing flow. Two findings were identified in it during this review. A complete security assessment of the protocol requires the orchestrator to be explicitly in scope.
- **Establish a meaningful test suite before the reaudit.** As documented in Finding [3.2](#), the Bridge Solidity contract, the Cosmos bridge module, and the orchestrator core have zero automated tests. Remediating this before a reaudit enables faster, deeper analysis and provides evidence that fixes to existing findings did not introduce regressions.
- **Remediate all findings identified in this report.** Remediate the findings, and provide fix commits or PRs for each so a reaudit can verify the remediations directly.

As a longer-term investment in code quality, we encourage continuing to strengthen security-focused development workflows, including the following:

- Comprehensive test coverage with end-to-end tests that exercise critical paths and edge cases
- Clear documentation of security assumptions, invariants, and known limitations
- Structured code-review processes that explicitly consider security implications

A reassessment following these improvements would allow for deeper analysis and provide

greater assurance for production deployment.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.